

MILTANK — the search player

Named for the classic Rollout user. `engine/rollout_leaf.js` is the mechanism; MILTANK is the player that uses it.

1. What it is, and why it is not MAG

ABRA has two players and until 2026-08-03 only one of them had a name.

MAG (`engine/magnemite.js`) is a fitted linear policy over `board.js` features, trained to predict **what a human clicks**. It is an imitator. Given a position it scores every legal option against a weight vector fitted on 60,000 corpus decisions and takes the best — one step, no lookahead, no notion of what happens next.

MILTANK decides by **playing the position out**. It forces a candidate pair of clicks, lets the game run to a conclusion, and takes the line that wins most. It has no fitted weights of its own and no opinion about what a person would do. It can therefore prefer a move that appears nowhere in the corpus, and it routinely does.

They are not rivals and MILTANK is not a replacement. The shipped arrangement is:

*MILTANK searches. When the search **cannot separate its options** — the finalists fall inside one standard error, or the position is already decided — it hands the decision back to MAG.*

That fallback is the point rather than a hedge. An argmax over estimates that overlap is a coin flip wearing a search's clothes, and a fitted human prior is a better tiebreak than dice. **Imitation is the floor; search is the ceiling.**

2. Where MILTANK decides

Decision	Before	Now
Which 4 to bring, which 2 to lead	<code>return 'default'</code> — packed order	90 brings enumerated and played out
Both clicks each turn	MAG's argmax	successive halving over every legal pair
Post-KO replacement	one-step heuristic	each candidate played out

The preview one matters most. Showdown's `RandomPlayerAI.chooseTeamPreview` returns the literal string `default`, `magnemite` never overrode it, so **every game ever played against this bot had its lead decided by where a Pokémon happened to sit in a team string** — on what `prior_player.js` itself calls "the single largest branch in the game".

3. The academic position, and where MILTANK sits in it

3.1 This is a simultaneous-move game, and MILTANK does not solve it

Pokémon is not turn-based. Both players commit without seeing the other, which puts it in the **simultaneous-move** class where a deterministic best response is exploitable by construction — the matrix game at each node can have no pure equilibrium.

The literature offers several MCTS variants for this setting: **Decoupled UCT (DUCT)**, sequential UCT, Exp3, and regret matching ([Lanctot et al., CIG 2014](#); [Tak, Lanctot & Winands](#)). DUCT lets each player pick from their own statistics without modelling the other. It is known **not to converge to an optimal strategy even in a single-state game** — and it still performs best empirically across their test set.

MILTANK is weaker than all of these and the docs should say so. It computes a *best response to a fixed opponent policy*: the opponent plays `chooseAction` during the stepped turn, identically for every candidate. That makes the comparison across candidates like-for-like, which is what the measurement needs, but it is not an equilibrium and it is exploitable by anyone who models it.

It is the right first cut for one stated reason: if a best response does not diverge from MAG, a mixture over the same cells will not either. `data/rollout-r3.json` records 72.9% divergence on 70 decisions, so it does diverge, and the equilibrium version is worth building.

3.2 The playout is fully random, and that is deliberate

The single most counter-intuitive result in this literature is that **a stronger playout policy can make the search worse**. Policies that maximise MCTS performance as playouts are *weak* at playing the game ([Are Strong Policies Also Good Playout Policies?](#), [AIIDE; An Analysis of Monte Carlo Tree Search](#)). What matters is how a policy *ranks* actions relative to each other, and **low-variance policies are dangerous** — a deterministic greedy playout replays the same line from one position, so N samples carry barely more information than one.

ABRA measured exactly this pathology before reading the literature. `chooseAction` is deterministic greedy; 53% of positions landed in the 0–10% or 90–100% bin and those bins were wrong by 22–29 points, and accuracy was **flat in N**. R1 then found a fully random playout judges a position at **68.18%** against **64.42%** for the greedy one.

So `--rollout-explore` defaults to `1.0`. This is not "we could not be bothered to write a good playout" — it is the fix the literature prescribes, and the greedy playout is the version that was tried and beaten.

Caveat, measured 2026-08-03: that result is about *judging a position*. Judging and *choosing an action* are different jobs, and a random opponent never punishes a wasted turn. A direct test on 40 corpus positions found Protect scoring −4.3pt at explore 1.0 and −5.2pt at 0.0 — i.e. the randomness is **not** obviously overpricing tempo. That is one measurement on one mechanic and it should not be read as settling the question.

3.3 The picker is a fixed-budget best-arm identification problem

A turn offers up to ~90 legal pairs. Estimating each and taking the max is **not** the same as finding the best one: the max over K noisy estimates is biased upward by roughly the spread of the noise, so the winner is disproportionately the arm that got lucky. At n=120 a win rate near 0.7 carries a standard error around 4

points, and the max over 63 such arms is inflated by ~ 2 SE of pure dice — larger than most real differences between two reasonable clicks.

The symptom was visible in play: **the bot clicked Recover at full HP**, a move MEDICHAM correctly heals 0 with. It was not choosing Recover; it was choosing whichever arm rolled well.

This is textbook **fixed-budget best-arm identification**, and the standard remedy is **successive halving**: spend the budget where it discriminates, not uniformly ([sequential halving / fixed-budget BAI](#)). MILTANK screens every pair at $N/3$, keeps the top 8, and re-tests those at $2N$ **with fresh seeds** — so an arm that advanced on lucky dice has to roll them twice. It is also cheaper than the flat version it replaced: $63 \times 40 + 8 \times 240$ beats 63×120 .

3.4 On-line policy improvement

The whole shape — take a base policy, improve it at decision time by rolling out — is Tesauro and Galperin's on-line policy improvement ([arXiv:2501.05407](#)). MAG is the base policy; MILTANK is the improvement operator. That framing predicts the fallback behaviour: where the rollout has no signal, the improved policy should return the base policy, which is what the noise-floor check does.

3.5 What the field says about the domain

[VGC-Bench](#) puts the team-configuration space at $\sim 10^{139}$, larger than Chess, Go, Poker, StarCraft or Dota, and ships 700,000 human battle logs with heuristic, LLM, behaviour-cloning and multi-agent-RL baselines. Two findings bear directly on ABRA:

- In **single-team mirror matches** their methods beat a professional VGC competitor. Strength in a narrow team distribution is achievable; that is roughly MILTANK's situation, playing a fixed pool of 230 open-sheet teams. 2. The agent that is best on **one** team is **more exploitable and generalises worse** across teams. This is a warning aimed squarely at ABRA: a bot tuned against the 230-team pool should not be assumed to hold up outside it, and any strength claim has to name the team distribution it was measured on.

4. Open team sheets are what make the preview search possible

MILTANK's bring/lead search is only meaningful because `|showteam|` hands over **the opponent's entire team with full sets**. A fitted bring prior can say "*Whimsicott leads 53% of the time*" — a fact about the population, not about this game. The sheet is information no usage prior can encode.

The opponent's own bring is **marginalised, not assumed**: each playout samples one of their 15 possible fours. Fixing a guess would optimise against one opponent out of fifteen and call it a plan.

Known weakness: that sample is *uniform*. A real opponent is not equally likely to bring every four. Weighting it by `bring_priors` or by which four best answers MILTANK's own team is unbuilt.

5. What MILTANK is not

- **Not an equilibrium.** Best response to a fixed policy (§3.1). Exploitable by design.

- **Not a tree search.** One step of forced actions, then a playout. No selection, no backprop, no tree. Calling it MCTS would overclaim.
- **Not measured to be stronger than MAG.** R1 (leaf accuracy), R2 (cost), R3 (divergence) all pass. **R4 — the SPRT against shipped greedy — has never been run.** Every claim in this document is about mechanism, not about winning more games. As of 2026-08-03 the bot's record against its author is 0–8.

6. Files

File	Role
<code>engine/rollout_leaf.js</code>	<code>rolloutWinProb</code> , <code>rolloutAfterActions</code> — the leaf
<code>engine/mag_bot.js</code>	MILTANK's three decision points; <code>--miltank</code> (<code>--rollout</code> aliases it)
<code>engine/medicham2-browser.js</code>	the doubles engine the playouts run in
<code>engine/rollout_r1/r2/r3.js</code>	the gates
<code>docs/ROLLOUT-design.md</code>	the original design note

Flags

Flag	Default	Meaning
<code>--miltank</code>	off	use the search player
<code>--rollout-n</code>	200	playouts per candidate at the final stage
<code>--rollout-explore</code>	1.0	playout randomness — see §3.2 before lowering it
<code>--rollout-turns</code>	60	horizon before a playout is scored
<code>--preview-n</code>	40	playouts per candidate bring
<code>--preview-ms</code>	15000	preview deadline; whatever was scored by then is reported

Sources

- [Monte Carlo Tree Search Variants for Simultaneous Move Games — Lanctot et al., CIG 2014](#)
- [MCTS in Simultaneous Move Games — Tak, Lanctot & Winands](#)
- [Are Strong Policies Also Good Playout Policies? — AIIDE](#)
- [An Analysis of Monte Carlo Tree Search](#)
- [Fixed-Budget Best-Arm Identification in Structured Bandits](#)
- [On-line Policy Improvement using Monte-Carlo Search — Tesauro & Galperin](#)

- VGC-Bench: Towards Mastering Diverse Team Strategies in Competitive Pokémon